

10 simple rules for making your software outlive your job

Richard Littauer^{1*}, Greg Wilson², Jan Ainali³, Eman Abdullah AlOmar⁴, Sylwester Arabas⁵, Yanina Bellini Saibene⁶, Kris Bubendorfer⁷, Kaylea Champion⁸, Clare Dillon⁹, Jouni Helske¹⁰, Pieter Huybrechts¹¹, Daniel S. Katz¹², Chang Liao¹³, David Lippert¹⁴, Fang Liu¹⁵, Pierre Marshall¹⁶, Daniel R. McCloy¹⁷, Ian McInerney¹⁸, Mohamed Wiem Mkaouer¹⁹, Priyanka Ojha²⁰, Christoph Treude²¹, Ethan P. White²²

- 1** CUIOSS, SustainOSS, GNOME Foundation, and Te Herenga Waka Victoria University of Wellington, Te Whanganui-a-Tara Wellington, Aotearoa New Zealand, 0000-0001-5428-7535
 - 2** Third Bit, Toronto, ON, Canada, 0000-0001-8659-8979
 - 3** Open By Default, 0000-0001-8747-1670
 - 4** Department of Systems and Enterprises, Stevens Institute of Technology, Hoboken, NJ, USA, 0000-0003-1800-9268
 - 5** AGH University of Krakow, Poland, 0000-0003-2361-0082
 - 6** rOpenSci, Santa Rosa, LP, Argentina, 0000-0002-4522-7466
 - 7** Te Herenga Waka Victoria University of Wellington, Wellington, New Zealand, 0000-0003-4315-8337
 - 8** Computing and Software Systems, University of Washington Bothell, Bothell, WA, USA, 0000-0001-6196-942X
 - 9** CUIOSS, Lero, and University of Galway, Ireland, 0009-0008-6205-0296
 - 10** INVEST Research Flagship Centre, University of Turku, Finland, 0000-0001-7130-793X
 - 11** Research Institute for Nature and Forest (INBO), Brussels, Belgium, 0000-0002-6658-6062
 - 12** University of Illinois Urbana-Champaign, IL, USA, 0000-0001-5934-7525
 - 13** Atmospheric, Climate, and Earth Sciences, Pacific Northwest National Laboratory, Richland, WA, USA, 0000-0002-7348-8858
 - 14** Open Source Program Office, George Washington University, Washington, DC, USA, 0009-0003-6444-9595
 - 15** Georgia Institute of Technology, GA, USA 0000-0002-3383-2191
 - 16** Oxford University, UK, 0000-0001-9245-7670
 - 17** Institute for Learning & Brain Sciences, University of Washington, Seattle, WA, USA, 0000-0002-7572-3241
 - 18** Department of Mechanical Engineering, Imperial College London, London, UK, 0000-0003-2616-9771
 - 19** Department of Computer Science, University of Michigan-Flint, Flint, MI, USA, 0000-0001-6010-7561
 - 20** 0000-0002-6844-6493
 - 21** Singapore Management University, Singapore, 0000-0002-6919-2149
 - 22** Department of Wildlife Ecology and Conservation, University of Florida, Gainesville, FL, USA, 0000-0001-6728-7745
- * richard@burntfen.com

We are grateful for contributions from Tobias Augspurger, Bill Branen (0000-0002-4735-6624), Paola Corrales (0000-0003-1923-9129), Sam Cunliffe (0000-0003-0167-8641), David Eysers (0000-0002-7284-8006), Elena Findley-de Regt, Tommy Guy (0009-0003-3652-5036), Jonathan Guyer (0000-0002-1407-6589), Mala

Kumar (0009-0004-3619-1037), Geoffrey Lentner (0000-0001-9314-0683), Georg Link (0000-0001-6769-7867), Daniel Morillo-Cuadrado (0000-0003-3021-3878), David Pérez-Suárez (0000-0003-0784-6909), Phani Velicheti (0009-0004-2580-3624), and everyone else who provided feedback along the way.

Abstract

Loss of key personnel has always been a risk for research software projects. Key members of the team may have to step away due to illness or burnout, to care for a family member, from a loss of financial support, or because their career is going in a new direction. Today, though, political and financial changes are putting large numbers of researchers out of work simultaneously, potentially leaving large amounts of research software abandoned. This article presents ten rules to help researchers ensure that the software they have built will continue to be usable after they have left their present job—whether in the course of voluntary career moves or researcher mobility, but particularly in cases of involuntary departure due to political or institutional changes.

Introduction

Academic life is often uncertain, particularly for those who work on research software, an asset that is essential to modern research [1] but often underfunded, undercited, and overlooked [2]. In some countries, the situation has recently worsened dramatically due to changes in the political and economic climate, and in others such instability is long-standing. In such situations, researchers in all disciplines may face the loss of funding or jobs, a smaller pool of incoming students, and institutional policies that prioritize commercial returns. Researchers in non-academic organizations such as government laboratories, non-governmental organizations (NGOs), and non-profits are also affected [3] as national research strategies are changed and associated public funding is redirected or cut entirely [4–8].

Ideally, research software projects should be *resilient* to such societal instability. Historically, open-source research software has been longer-lived than other kinds of open-source software and government involvement has been associated with longer lifespan [9]. Unfortunately, this may mean that the challenges of responding to macro-level changes in software development support are new and unfamiliar to research software developers, and they may not have put structures in place to ensure that their code will outlast situations such as changing workplaces or careers, partnerships failing, a lab closing down, or tooling and digital infrastructure becoming unavailable. We therefore present ten rules to ensure that your software remains usable by others even if you are no longer able to work on it. Some of these rules are most useful when adopted at the beginning of a project to build resilience over time (e.g., Rules 1, 2, 5, and 7), while others are particularly relevant when a transition or disruption is imminent (e.g., Rules 3 and 8). You do not need to adopt all of them to make things better; instead, as in any crisis situation, you should do what you can, where you are, with what you have.

This guide focuses on software. Other guides already exist for data [10], where archives such as Zenodo, FigShare, the End Of Term Archive or the Internet Archive are now widely used to ensure long-term availability. However, we have not found any guide to make research software resilient or to *sunsetting* it (i.e., phasing out development and support gracefully). Even if your job is not in danger, these rules will help others access and use your code, contribute to your project, and to reproduce and cite your work. As such, our recommendations are aligned with the global push for a more equitable assessment of research contributions by initiatives such as DORA,

Rule 1: Stay within the law

35

Ensuring your work remains usable is not worth putting yourself at legal risk. Before following any of the rules below, make sure you have a legal right to do so. Most institutions and journals now have policies for licensing code and/or releasing it publicly [11,12]. Funding agencies at various levels also often have policies (e.g., EU¹, UKRI², and NASA³) which may or may not align with those of institutions and journals, and institutional policies may place restrictions on what you are allowed to say publicly about your situation. Our first actionable rule is therefore to find out what those policies are, e.g., whether you need explicit permission to publish your software, and if so, from whom.

If you are unsure whether there is a policy or not, ask your direct manager, the person who pays you, or offices like the research office, the library, or the technology transfer office. If your institution has a dedicated Open Source Program Office (OSPO), ask them or connect to networks like CURIOS, the OSPO Alliance, or the TODO Group that advocate for OSPOs. If you work with multiple institutions, look at your contract and the bilateral agreements around products or deliverables.

Once you know whether your institution has a policy, get whatever sign-offs you need immediately. Reach out to colleagues who can review your code if that is necessary for publishing it, and review theirs if asked in return. If it appears that there is no formal sign-off process, send an email to someone in authority (e.g., the chair of your department or your grant officer) saying that you believe this to be the case, and copy that email to an account you will be able to access after leaving your institution.

Do not assume that if you had permission before, you have it now or will have it in the future. Policies are changing rapidly, and you may find yourself locked into one that no longer allows you to do what you want. Also consider that the person you report to may be replaced with one who knows you less well or is less sympathetic, so acting now may be easier than acting later.

Finally, if you have your next job lined up, ask about their policies and make sure that your right to share your work is written into your contract. Making your code comply with policy after the fact is riskier and more time-consuming than doing so early, and sometimes not possible.

Not all rules are created equal

66

Anyone who has worked in a large bureaucratic organization knows the difference between the rules as written and the rules as enforced. If you are leaving your position on short notice or under difficult circumstances, you may want to be selective about which steps you take proactively and which you just haven't gotten around to yet. Members of marginalized groups have often had more practice with these tactics than members of privileged groups [13]; if you belong to the latter, a quiet conversation or two with selected colleagues may help you see your priorities more clearly.

74

¹https://web.archive.org/web/20250424181252/https://commission.europa.eu/about/departments-and-executive-agencies/digital-services/open-source-software-strategy_en (archived 2025-04-24)

²<https://web.archive.org/web/20250422025105/https://www.ukri.org/manage-your-award/publishing-your-research-findings/making-your-research-data-open/> (archived 2025-04-22)

³<https://web.archive.org/web/20250422005753/https://science.nasa.gov/open-science/nasa-open-science-funding-opportunities/> (archived 2025-04-22)

Rule 2: Define your threat model

When making plans, it's useful to know what you're planning *for*. A **threat model** is a structured analysis of where threats might come from, what form they might take, what social and technical vulnerabilities they might exploit, and how they might be mitigated or resolved [14]. Being explicit about threat models helps you prioritize, build consensus with colleagues, and check whether you have forgotten something important. The Open Worldwide Application Security Project (OWASP) provides definitions, guidance, and resources for software threat modeling through the Threat Modeling Project website.⁴ As with all crisis planning, making plans before you need them may help you identify risks you can eliminate proactively.

1. **Individual threats** affect one or a few members of your team, such as an international student having their visa revoked without notice, or a contributor taking extended leave. The most common way to prepare for this is to require everyone to document their work, but that rarely works in practice:

- (a) The hours spent writing those descriptions are hours *not* spent doing research, so people will always short-change the former to focus on the latter.
- (b) People invariably fail to write down the “obvious” parts of their work that are anything but obvious to the next person.

Rule 8 explores alternatives, particularly ones that can be put in place on short notice.

2. **Leadership threats** are individual threats that affect the project's leader. As the public face of a project, they may be more susceptible than other project members to threats such as doxxing or being personally targeted in the media because of their work. Rule 3 discusses ways to plan for this.
3. **Institutional threats** affect large groups at once, such as your department being shut down or your entire field having funding cut. These events affect so many people at the same time that the rest of the community cannot absorb them. Regional and national governments plan for disasters like these by having evacuation plans to get victims to safe(r) places and provide them with emergency shelter and supplies. When government institutions themselves are in a period of upheaval, mutual aid and civil society organizations may serve as an alternative. At the time of writing, universities and researchers' professional societies have not started to externally share equivalent planning, if they have started making those plans at all.
4. **Global threats** affect everyone, not just researchers. For example, there is no technical or legal obstacle for the US government requiring American companies to charge for video conferencing calls involving participants outside the United States. Similar levies on email, file storage, and other online services would undoubtedly prompt national governments to find alternatives, but would still result in (at least) months of disruption.

When performing a threat analysis, it is okay to decide that *for your project* there are certain kinds of threats that you do not have a plan for (or the plan is “abandon the project until further notice”). Making those decisions before the threat arises means one less thing to cogitate on during the crisis, in the unfortunate case where threat does materialize.

⁴<https://owasp.org/www-project-threat-modeling/>

Rule 3: Decide if you are ending, pausing, or handing off

Thousands of books describe how to start a business, but only a handful discuss handing one over or winding one down, and most of those focus on succession within family-owned businesses. Guides to running a laboratory say little more [15, 16], so such wisdom as we have is passed on person-to-person if it is passed on at all. Consider the following augmented Software Lifecycle (SLDC), that is based on [17] and modified for this paper [Insert Figure 1].

The first step is to decide if work on the project is ending, being suspended temporarily (for some version of “temporarily”), or if you are handing it on to someone else. If the project is ending or going on hiatus, focus on making what you have accomplished findable and citable as discussed in the rules that follow. To help you prioritize, write up the goals you initially had for this project and ask how well they were served by what you actually did. This approach could lead you to understand that the project does not have to conclude, but can be integrated into someone else’s work, even without your continued participation.

If instead you hope to hand the project on to someone else, you can either choose a successor (who will typically be someone already working on the project) or ask your peers for volunteers or recommendations. Writing a *succession plan* that describes what the work will actually entail will help in both cases. What will your successor have to learn and do during the transition? What will they be responsible for once they are in charge? And crucially, how much work is running the project actually going to be? Please be honest rather than optimistic: finding out after you have agreed to take something on that it is much larger than you realized can strain both professional and personal relationships.

If you’re not currently facing an abrupt transition in project leadership, developing a *software management plan* now can help clarify the roles and responsibilities of running a project, making it easier to reason about filling those roles when a decision about succession, suspension, or abandonment becomes necessary. The Software Sustainability Institute has resources⁵ to guide the development of software management plans.

Some projects try to ensure longevity by working with a fiscal sponsor such as NumFOCUS, Software Freedom Conservancy, the Eclipse Foundation, Open Source Collective, or other foundations and fiscal hosts. Through fiscal hosting, a project can take donations from its community to fund continued maintenance and other costs. A fiscal sponsor gives the software a neutral home where it is no longer owned by a single research organization, which can help to insulate it from changes in policy. Fiscal sponsors also tend to have lower overhead and procurement costs compared to university or government entities.⁶

However, fiscal sponsorship is only an option in some countries, and being accepted as a sponsored project is a long and challenging process. Even when it is possible, the copyright holder of the work may continue to be the institution that paid for the development of the software. Private foundations may be a more attractive alternative in some jurisdictions; as per Rule 1, check with *local* legal experts before taking action.

⁵<https://www.software.ac.uk/guide/writing-and-using-software-management-plan>

⁶That said, if your institution has an Open-Source Program Office (OSPO), that can be a source of stability, for example through small internal grants or through connections to a network of possible future co-maintainers or successors for your project.

Rule 4: Choose an open license (if you can)

Making code publicly available does not ensure that other people can use it; you maintain all copyright unless you explicitly include a license, so they may still need explicit permission, or may simply be unsure whether they need permission or not. The Open Source Initiative (OSI) maintains a list of open source licenses it has approved; the implications of these licenses are widely understood, so choosing one of those will make your project more understandable to others. Please do *not* try to write your own license or simply waive copyright and place the software into the public domain, as what is meant by “public domain” varies from country to country. (In countries such as France and Germany it may not actually be possible to waive copyright and place a work in the public domain.)

Broadly speaking, the MIT, Apache 2.0, or BSD-3-Clause licenses are *permissive* licenses that place the fewest restrictions on modification, distribution, and re-use. The GPLv3 and AGPL-3.0 licenses are *copyleft* licenses that require people to share their modifications to the software with the community [18]. These copyleft licenses can prevent companies from taking advantage of work without giving anything in return, but some institutions discourage or disallow their use out of fear that they will constrain commercialization, and some companies have policies against using software with some copyleft licenses. Note also that if you hope to someday commercialize the research software you have written, consider carefully whether an open-source license will interfere with that eventuality.

More broadly, Creative Commons licenses can be used for documentation or research reports but should be avoided for actual code, while ethical licenses or licenses based on the Blue Oak Model License may also be appropriate. choosealicense.com and other sources provide guidance [19,20]; discussion of why people *don’t* share is also helpful [21]. Whatever license you choose, place its text in a file called `LICENSE`, `LICENSE.txt`, or `LICENSE.md` in the root directory of your project, as this is where people and automated tools alike will look for it.

The law, again

Most discussion of open source licensing focuses on circumstances in a handful of affluent countries in the same way as discussion of fiscal sponsorship (Rule 3). For example, many Latin American countries have access to information laws that include research outcomes and artifacts. You can publish data and software under that, but the institution sets the license type, and you must use institutional repositories. As always, consult *local* legal experts before taking action.

Rule 5: Save everything in multiple places

Now that you are legally able to share your code, remember LOCKSS: Lots of Copies Keep Stuff Safe.⁷ But copies are not enough: the threats to your project may be political as well as technological (Rule 2), so you should ensure that loss of institutional support (or worse, that institution turning on you) does not mean loss of access.

The road not taken

Fifteen years ago, a file-sharing system for scientific data based on the BitTorrent protocol failed to find wide adoption because of the (quite reasonable) association in many institutions’ collective minds between

⁷A related mnemonic is “3-2-1”: 3 copies, on 2 different media, 1 in a different location.

BitTorrent and illegal downloading [22]. At the time of writing, though, many individuals and groups are turning to BitTorrent to share datasets that are at risk of disappearing because of the protocol’s resilience.

GitHub, GitLab, and BitBucket are social coding platforms centered around Git, a widely-used open source version control tool. However, they are all commercial entities based in the same legal jurisdiction, which means they are vulnerable to *correlated threats*: trouble with any of them may mean trouble with all of them. Codeberg, while currently much smaller, is a non-profit based in a different jurisdiction; consider mirroring your work there or using it as your primary host. Remote Git repositories can also be self- or institution-hosted; these typically lack the social and interactive features of the commercial options, but (depending on the institution or private host’s location/jurisdiction) may provide some protection against the aforementioned correlated threats.

Keep in mind that persistent code archival is not a feature of development platforms. Repositories can be altered, made private, or deleted (also as a result of government takedowns⁸); the platform itself may be shut down (as happened to the non-profit Gna!⁹), or it may change its revenue model (as happened with SourceForge¹⁰, which once dominated the open source repository space in the way that GitHub does today [23]).

When you create projects on hosted services, use a team account as the project owner rather than a personal account. Doing this makes it easier to give other people permission to manage the project, and as noted in Rule 6, a project with multiple owners from different institutions is harder for any one institution to lock down. For the same reason, do not rely solely on logins or email addresses associated with your institution: instead, ask yourself if you will still have access to the project if you lose that identity, and add an identity you personally control to the team that owns the project. You can also add your ORCID to the project’s metadata so that the project links to a profile that you control even when your contact address changes.

What’s in a project?

When deciding what to store where, remember that your version control repository only stores part of what makes up your project. For example, while GitHub wikis are stored in ordinary git repositories, the issues and discussions live in GitHub’s internal database; you can use the GitHub API to download their content, but (a) the result is intended for consumption by machines, not people, and (b) if you are doing this on short notice, the odds are high that other people are trying to do it at the same time. GitHub’s servers may not be able to manage that load when you need them to, so use tools like offline-issues to save hosted content as plain-text files regularly.

Version control is not the only way to create and save copies of your work. Software Heritage archives software from multiple forges; you can also snapshot the current state of repositories in a compressed archive file (e.g., `.zip` or `.tar.gz`) and deposit those copies with the Open Science Foundation (OSF), Zenodo, or Figshare. (Zenodo even integrates with GitHub so that tagging a release on GitHub automatically triggers deposit of a new archive on Zenodo, but again, this automation is only useful as long as both ends are accessible.) Institutional, national, or international data repositories also enhance longevity.

⁸https://github.com/github/transparency/tree/main/data/government_takedowns

⁹<https://en.wikipedia.org/wiki/Gna!>

¹⁰<https://en.wikipedia.org/wiki/SourceForge>

Storing copies on someone else’s computer is not the only option: you can (and should) make copies of your projects on computers that you own. Again, this is a place where colleagues can help: ask them to make copies on their computers in exchange for you making copies of theirs on yours. Whatever you do, document it as rOpenSci has.

Similarly, if you distribute your code as a package on PyPI, Conda, or CRAN, that package can contain all of the source code. Doing this also fosters community collaboration. When setting these up, you should ideally ensure that control over the distribution channels for a package is not limited to a single developer. However, this is not always possible. For example, while PyPI actively encourages this, CRAN policies allow only single point of contact.

Rule 6: Encourage community adoption

Publishing your code is not the same as publicizing it. The more people who know about and rely on your project, the greater the chances that someone will help keep it alive, whether by working on it directly or supporting you in doing so. If you have not named your project yet, choose a name that does not hint at direct and exclusive affiliation to a single institution. This can help if the code needs to be relocated, and also makes contributors from other institutions feel welcome. Additionally, label entry-level issues (e.g., to use the “good first issue” label on GitHub) so that people can easily find a place to start [24].

Going further, there is a causal link between social media mentions and both new adopters and new contributors [25]. Announcing your project on mailing lists, forums, or social media, and talking about it at conferences are therefore necessary but not sufficient: since everyone else is doing this too, it is hard to be heard above the noise.

There are many good guides to what you can do to promote your project [26,27]. A short video showing how to use the software or a slide deck that others can incorporate into their lectures will increase uptake by lowering the cost of adoption. Similarly, a one-page website that opens with an elevator pitch explaining who the software is for and how it will improve their lives is much more likely to lead to that crucial “second glance” than a list of publications.

Disseminating your code through tutorials at conferences is another effective strategy. Tutorials are an opportunity for you, as an author, to describe your work in depth and (more importantly) convince your audience to use it. Networking through such events will enable you to build relationships with the people most likely to care about your project, and it is the best way to ensure that you land well after you leave your current position. It can also help to familiarize yourself with *other* tutorial resources that your (future) users or contributors might need. For example, point users to resources such as Software Carpentry¹¹ to learn the basics of command-line shells *prior to your tutorial or workshop* so that you can spend more of that time teaching them about your software.

Deeper, mentored engagements with your software like sprints, class projects, or Google Summer of Code¹² can go even further toward preparing someone else to co-maintain your software or take the reins when you step down. The obvious trade-off is that (compared to a workshop) mentorship is a larger investment of time in a smaller number of people. If you have students or post-docs under your direction, mentoring them in research software development and maintenance alongside your scientific mentorship can also increase your project’s chances of longevity.

Bring people in

¹¹<https://software-carpentry.org/>

¹²<https://summerofcode.withgoogle.com/>

If you can, have at least one person outside your organization commit to your code. Such community contributions lead to joint ownership of intellectual property (IP), which makes it harder for any single institution to lock down your work. You can reinforce this by adding their name to the copyright statement in your license.

Finally, remember that it is not all about *your* project. If you ask others to help maintain your code, be prepared to give them something in return: a letter of reference, help testing or documenting their project, or co-authorship credit on the project and associated publications.

Rule 7: Do what you’re supposed to (if you have time)

Research software is increasingly recognized as a citable object [28–30], and numerous books and research papers describe how to organize and run a research software project. Rather than repeating what others have said [31, 32], this rule is a prioritized checklist of things to do *if you are not facing immanent involuntary changes to your employment*. If you are, please consult the next rule.

1. Create a DOI for each release of your software to ensure citability and retrievability even if the project repository becomes unavailable, and add a Citation.CFF or codemeta.json file to your code [33] containing citation metadata. Tools such as cffinit, codemeta generator, and others can help you do this.
2. Submit your code for peer review and publication in a venue such as the Journal of Open Source Software, the Journal of Open Research Software, the Journal of Statistical Software, the R Journal, rOpenSci, or pyOpenSci.
3. Document how to *use* your software so that others can pick it up and use it if you are not available. In particular, add a README.md at the root of your repository that explains the purpose, setup, and use of your work, and add a tutorial showing how to use the major components of the code [34–38].
4. Document any datasets your project depends on, and use CSV, JSON, Parquet, HDF5, or other widely-recognized formats rather than proprietary formats. Make the data public (if allowed) and easy to access by your users, either by automating its download or incorporating the data in your project if possible. If that is not possible for legal reasons or because of the data’s size, include URLs with data version IDs (or the date of the last download) in your documentation.
5. Describe your project’s dependencies in a machine-readable way using Python’s pyproject.toml file, an npm package.json file, the DESCRIPTION and NAMESPACE files of an R package, a software bill of materials (SBOM), or whatever else is standard for your language.
6. Write at least a few tests for your software so that people who want to use it (or contribute to it) can tell if it is working in their environment. These do not have to be formal unit tests [39]: even a handful of “this input should produce this output” checks can save a lot of frustration [40]. Including sample inputs is also helpful, and removes dependence on external data sources which may themselves face continuity challenges.
7. Document how to *contribute* to your software, e.g., how to set up a development environment, run tests, and add new features. Such a guide leads to an increase in contributions [41], as does an explanation of the project’s governance, i.e., how decisions are made and who gets a voice in making them.

The above suggestions will help to reduce your project’s *technical debt*, but are not a panacea. Using descriptive variable names, adding in-line documentation, refactoring near-duplicated code into reusable functions, adopting automated code linting, setting up continuous integration testing, and many other widely-touted best practices will help make your project more resilient by making the software less effortful to maintain. Note that all of the suggestions above will also help with adoption, for the same reason that a tidy restaurant attracts more customers than a messy one. In addition, if you follow the suggestions above when you have time, there will be less for you to do when you don’t (as discussed in our next rule).

Rule 8: Do what you can (if you have to act quickly)

“Good enough” practices in research computing, such as creating .zip archives of your project as an alternative to using version control, may be more appropriate than “best practices” in an emergency [42]. (By analogy, a paramedic is not a doctor who cuts corners, but rather someone who specializes in working under very different circumstances.) If you are short on time:

1. Explanations of what the software *does* and *how to use it* take priority over documentation of its internals because (a) someone who knows the first two will be better able to figure out the third and (b) AI code assistants can do a reasonable job of helping newcomers navigate an unfamiliar code base if they are given enough context.
2. Similarly, a short screencast showing how you use your software is often faster to create than pages of documentation, and may actually be more useful if closed captions are attached to make the content searchable. Again, AI tools can help create those captions, though some editing is invariably required to clean up mistranscription of technical jargon.
3. If you do not have time to bring your project in line with best practices, try instead to clean it up by removing files that are no longer used, deleting sections of your setup instructions that no longer apply, closing bug reports that are no longer relevant, and so on. Eliminating clutter in this way will make what is useful easier to find.
4. Briefly document what happened. If you are the main developer working on a grant and you have moved on to something else, say that in the README. If the funding changed, then say that the work is unfunded. This information is often the most useful for users but the hardest to find. Saying it up front will help others understand how your work is positioned.
5. Following on from Rule 5, you can search the Software Heritage Archive to check if your repository has been saved. If not, submitting it is a simple process.

Rule 9: Talk about what you’re doing

All of the work that we have described is extra labor. You will not want to do it, particularly not when so many other things demand attention as well. Make that work meaningful by helping others. Tell your colleagues what you have learned about your institution’s legal requirements, about navigating its bureaucracy, and about the least hurtful way to break the news to your students, collaborators, and users. Share your own rules on social media (preferably Mastodon rather than X or Bluesky, as the latter

two are single points of institutional failure), and share things that *haven't* worked as well so that other people can avoid the same pitfalls.

Sharing your experience during a difficult time is valuable even if your only audience is yourself. Writing down what happened, what you did, how you felt, what you have learned, and what you hope for are evidence-based strategies for protecting your mental health in stressful times and for healing afterwards [43, 44].

Most importantly, take the time to grieve. Having years of work come to an end can take a toll on your mental health, even if it stems from a *voluntary* career transition. If your transition is triggered by institutional or societal factors, many of your colleagues may be suffering as well, and those who haven't been hit (yet) may be affected by survivors' guilt. Be angry, be sad, but remember that in the aftermath of a disaster—natural or man-made—people often become altruistic, resourceful, and brave [45].

Rule 10: Organize

There is little point surviving today's flood if you drown in tomorrow's. The best strategy is to take preventive action, and the best way to do that in a research context is to become active in professional associations (and/or labor unions, if your job class is affiliated with one) and push them to take meaningful action. Professional societies are often better placed than individuals to exert pressure on government officials, and revising a society's bylaws and budget to prioritize safety-net programs (such as bridge funding) for its members can be pursued as well.

Many readers of this paper will find the thought of doing this uncomfortable, in large part because voluntary organizations have seen a slow but steady decline in membership over several decades [46]. Although there are many contributing factors, the net result is that professionals aged 30–60 are less likely to be involved in civil society and the political process than the generation before them. This disengagement effectively hands power to extremists and special interest groups and enables them to seize control of civil institutions in order to advance their agendas [47]. To see the impact, ask yourself who is more likely to run for a seat on your local school board: someone with a doctorate in epidemiology or an anti-vax conspiracy theorist?

Postscript

In 2013, the government of Canada ordered the closure of five of the Department of Fisheries and Oceans' seven libraries [48]. No one knows how many thousands of volumes of irreplaceable environmental records were destroyed, but even scientists who were directly affected did not make plans in case something similar happened again. In contrast, when faced with the prospect of a populist demagogue becoming president, researchers in Argentina knew from their collective experience what might happen next and took action accordingly [49].

It is our hope that by documenting some recommendations based on the collective experience and wisdom of researchers across the globe, we may empower others to make their own software projects more resilient. However, we left many suggestions out of this paper due to space constraints or because we felt they would age poorly. Other ideas may be omitted because we have been lucky enough not to have to think about them. In an effort to correct that, we have set up a task force hosted by CURIOS, SustainOSS, and the Research Software Alliance, and plan to run monthly meetings to share ideas and support one another. If you have suggestions, please contribute them at <https://www.researchsoft.org/tf-succession-planning/>.

References

1. Pearson H, Ledford H, Hutson M, Van Noorden R. Exclusive: the most-cited papers of the twenty-first century. *Nature*. 2025;640(8059):588–592. doi:10.1038/d41586-025-01125-9.
2. Carver JC, Weber N, Ram K, Gesing S, Katz DS. A survey of the state of the practice for research software in the United States. *PeerJ Computer Science*. 2022;8:e963. doi:10.7717/peerj-cs.963.
3. Woodward A, Leeder S. Making science great again. Or not. *International Journal of Epidemiology*. 2025;54(2). doi:10.1093/ije/dyaf029.
4. *Nature*. Mexican science suffers under debilitating budget cuts. *Nature*. 2019;572:294–295. doi:10.1038/d41586-019-02332-x.
5. *Nature*. Brazil’s scientists face 90Nature. 2021;598:566. doi:10.1038/d41586-021-02882-z.
6. *Nature*. Science is under threat in Argentina — we must call out the danger. *Nature*. 2023;622:433. doi:10.1038/d41586-023-03228-7.
7. Inspireurope+. Researchers at Risk: An Update on National level Actions in Europe 2024; 2024. Available from: <https://sareurope.eu/inspireurope/inspireurope-publications-policy/>.
8. *Nature*. Trump 2.0: An assault on science anywhere is an assault on science everywhere. *Nature*. 2025;639(8053):7–8. doi:10.1038/d41586-025-00562-w.
9. Malviya-Thakur A, Milewicz R, Jahanshahi M, Paganini L, Vasilescu B, Mockus A. Scientific open-source software is less likely to become abandoned than one might think! Lessons from curating a catalog of maintained scientific software. In: *ACM International Conference on the Foundations of Software Engineering (FSE)*; 2025. Available from: <https://arxiv.org/abs/2504.18971>.
10. Perkel JM. How to make your scientific data accessible, discoverable and useful. *Nature*. 2023;618(7967):1098–1099. doi:10.1038/d41586-023-01929-7.
11. Katz DS, Niemeyer KE, Smith AM. Publish your software: Introducing the *Journal of Open Source Software (JOSS)*. *Computing in Science & Engineering*. 2018;20(3):84–88. doi:10.1109/MCSE.2018.03221930.
12. Ham D, Hargreaves JC, Kerkweg A, Roche DM, Sander R. The publication of geoscientific model developments v1.2. *Geosci Model Dev*. 2019;12(6). doi:10.5194/gmd-12-2215-2019.
13. Scott JC. *Weapons of the Weak: Everyday Forms of Peasant Resistance*. Yale University Press; 1987.
14. Torr P. Demystifying the threat modeling process. *IEEE Security & Privacy*. 2005;3(5):66–70. doi:10.1109/MSP.2005.119.
15. Barker K. *At the Helm: Leading Your Laboratory*. Cold Spring Harbor Laboratory Press; 2010.
16. Cohen CM, Cohen SL. *Lab Dynamics: Management and Leadership Skills for Scientists*. 3rd ed. Cold Spring Harbor Laboratory Press; 2018.

17. Yehudi Y, Cashman M, Felderer M, Goedicke M, Hasselbring W, Katz DS, et al. Towards Defining Lifecycles and Categories of Research Software. *deRSE25 Conference Proceedings*. 2025;doi:10.5281/zenodo.15002661. 478
479
480
18. Morin A, Urban J, Sliz P. A Quick Guide to Software Licensing for the Scientist-Programmer. *PLOS Computational Biology*. 2012;8(7):e1002598. 481
482
doi:10.1371/journal.pcbi.1002598. 483
19. Fogel K. Producing Open Source Software (updated); 2020. Available from: <https://producingoss.com/>. 484
485
20. Fortunato L, Galassi M. The case for free and open source software in research and scholarship. *Phil Trans Royal Soc A*. 2021;379(2197). 486
487
doi:10.1098/rsta.2020.0079. 488
21. Gomes DGE, Pottier P, Crystal-Ornelas R, Hudgins EJ, Foroughirad V, Sánchez-Reyes LL, et al. Why don't we share data and code? Perceived barriers and benefits to public archiving practices. *Proceedings of the Royal Society B: Biological Sciences*. 2022;289(1987). doi:10.1098/rspb.2022.1113. 489
490
491
492
22. Langille MGI, Eisen JA. BioTorrents: A File Sharing Service for Scientific Data. *PLoS ONE*. 2010;5(4):e10071. doi:10.1371/journal.pone.0010071. 493
494
23. Tamburri DA, Blincoe K, Palomba F, Kazman R. "The Canary in the Coal Mine..." A cautionary tale from the decline of SourceForge. *Software: Practice and Experience*. 2020;50(10):1930–1951. doi:10.1002/spe.2874. 495
496
497
24. Steinmacher I, Conte T, Gerosa MA, Redmiles D. Social Barriers Faced by Newcomers Placing Their First Contribution in Open Source Software Projects. In: *Proc. CSCW'15*. ACM; 2015. Available from: <https://doi.org/10.1145/2675133.2675215>. 498
499
500
501
25. Fang H, Lamba H, Herbsleb J, Vasilescu B. "This is damn slick!": estimating the impact of tweets on open source project popularity and new contributors. In: *Proc. 44th International Conference on Software Engineering*; 2022. p. 2116–29. Available from: <https://doi.org/10.1145/3510003.3510121>. 502
503
504
505
26. Kuchner MJ. *Marketing for Scientists: How to Shine in Tough Times*. Island Press; 2011. 506
507
27. Bellini Saibene Y, Salmon M. *Marketing Ideas For Your Package*; 2024. Available from: <https://ropensci.org/blog/2024/03/07/package-marketing/>. 508
509
28. Smith AM, Katz DS, Niemeyer KE, Group FSCW. Software citation principles. *PeerJ Computer Science*. 2016;2:e86. doi:10.7717/peerj-cs.86. 510
511
29. Katz DS, Chue Hong NP, Clark T, et al. Recognizing the value of software: a software citation guide [version 2; peer review: 2 approved]. *F1000Research*. 2021;9:1257. doi:10.12688/f1000research.26932.2. 512
513
514
30. Garijo D, Arroyo M, Gonzalez E, Treude C, Tarocco N. Bidirectional paper-repository tracing in software engineering. In: *Proceedings of the 21st International Conference on Mining Software Repositories*; 2024. p. 642–646. Available from: <https://doi.org/10.1145/3643991.3644876>. 515
516
517
518
31. Sandve GK, Nekrutenko A, Taylor J, Hovig E. Ten Simple Rules for Reproducible Computational Research. *PLOS Computational Biology*. 2013;9(10):e1003285. doi:10.1371/journal.pcbi.1003285. 519
520
521

32. Wilson G, Aruliah DA, Brown CT, Chue Hong NP, Davis M, Guy RT, et al. Best Practices for Scientific Computing. *PLOS Biology*. 2014;12(1):e1001745. doi:10.1371/journal.pbio.1001745.
33. Druskat S, Spaaks JH, Chue Hong N, Haines R, Baker J, Bliven S, et al.. Citation File Format; 2021. Available from: <https://doi.org/10.5281/zenodo.5171937>.
34. Lee BD. Ten simple rules for documenting scientific software. *PLOS Computational Biology*. 2018;14(12):e1006561. doi:10.1371/journal.pcbi.1006561.
35. Huybrechts P, Trekels M, Abraham L, Desmet P. B-Cubed software development guide; 2024. Available from: <https://docs.b-cubed.eu/guides/software-development/>.
36. Littauer R. Standard Readme; 2025. Available from: <https://github.com/RichardLitt/standard-readme>.
37. Katz DS, Forbes M, Silen L, Curcuru S, Hucka M, Tang Y, et al.. Open-source software project documents; 2025. Available from: <https://github.com/corsa-center/oss-documents>.
38. The Turing Way Community. The Turing Way: A handbook for reproducible, ethical and collaborative research; 2025. Available from: <https://doi.org/10.5281/zenodo.3233853>.
39. Irving D, Hertweck K, Johnston L, Ostblom J, Wickham C, Wilson G. Research Software Engineering with Python: Building Software that Makes Research Possible. CRC Press/Taylor and Francis; 2021. Available from: <https://doi.org/10.1201/9781003143482>.
40. Taschuk M, Wilson G. Ten simple rules for making research software more robust. *PLOS Computational Biology*. 2017;13(4):e1005412. doi:10.1371/journal.pcbi.1005412.
41. Sholler D, Steinmacher I, Ford D, Averick M, Hoyer M, Wilson G. Ten simple rules for helping newcomers become contributors to open projects. *PLOS Computational Biology*. 2019;15(9):e1007296. doi:10.1371/journal.pcbi.1007296.
42. Wilson G, Bryan J, Cranston K, Kitzes J, Nederbragt L, Teal TK. Good Enough Practices in Scientific Computing. *PLOS Computational Biology*. 2017;13(6):1–20. doi:10.1371/journal.pcbi.1005510.
43. Pennebaker JW, Smyth JM. Opening Up by Writing It Down: How Expressive Writing Improves Health and Eases Emotional Pain. 3rd ed. Guilford Publications; 2016.
44. Cullen L, Hanrahan K, Farrington M, Tucker S, Edmonds S. Evidence-Based Practice in Action: Comprehensive Strategies, Tools, and Tips From University of Iowa Hospitals & Clinics. 2nd ed. Sigma Theta Tau International; 2022.
45. Yang Y. The Effect of Disasters on Altruistic Behavior and its Mechanisms. *Journal of Education, Humanities and Social Sciences*. 2024;26:572–577. doi:10.54097/xng2q069.
46. Putnam RD. Bowling Alone: The Collapse and Revival of American Community. 2nd ed. Simon & Schuster; 2020.

47. Bueno de Mesquita B, Smith A. The Dictator's Handbook: Why Bad Behavior is Almost Always Good Politics. PublicAffairs; 2022. 565
566
48. Nikiforuk A. Harper government shuts down 'world class' collection on freshwater science and protection;. Available from: 567
<https://thetyee.ca/News/2013/12/09/Dismantling-Fishery-Library/>. 568
569
49. Del Boca P. Disorderly Transition: Preserving the Argentine Government's Open Data; 2024. presented at csv,conf 8. Available from: 570
<https://www.youtube.com/watch?v=8dUyi40YAdM>. 571
572